
Otimização de alocação de ativos e posicionamento de equipes de manutenção

— Matheus Cascalho dos Santos —
Pedro Goulart

Modelagem Matemática

$$\min f_1 = \sum_{i=1}^n \sum_{j=1}^m d_{ij} x_{ij}$$

$$\min f_2 = \max_k \left(\sum_{i=1}^n h_{ik} \right) - \min_k \left(\sum_{i=1}^n h_{ik} \right)$$

- 1) Cada equipe deve ser alocada a exatamente uma base de manutenção:

$$\sum_{j=1}^M y_{jk} = 1 \quad \forall k \in S$$

- 2) Cada ativo deve ser atribuído a exatamente uma base de manutenção:

$$\sum_{j=1}^M x_{ij} = 1 \quad \forall i \in N$$

- 3) Cada ativo só pode ser atribuído a uma base se esta estiver ocupada por pelo menos uma equipe de manutenção:

$$x_{ij} \leq y_{jk} \quad \forall i \in N \quad \forall j \in M \quad \forall k \in S$$

- 4) Cada ativo deve ser atribuído a exatamente uma equipe de manutenção:

$$\sum_{k=1}^S h_{ik} = 1 \quad \forall i \in N$$

- 5) Cada ativo só pode ser atribuído a uma equipe se esta estiver ocupando a base na qual o ativo estiver alocado:

$$h_{ik} \leq \frac{x_{ij} + y_{jk}}{2} \quad \forall i \in N \quad \forall j \in M \quad \forall k \in S$$

- 6) Cada equipe deve ser responsável por pelo menos $\eta \frac{n}{s}$ ativos, onde $\eta = 0.2$ é um percentual definido pela empresa:

$$\sum_{i=1}^N h_{ik} \geq \eta \frac{n}{s} \quad \forall i \in N \quad \forall k \in S$$

Estrutura de dados da Solução

Variáveis utilizadas:

- `ativo_equipe`: Mapeia cada ativo à sua equipe.
- `equipe_base`: Define a base de cada equipe.

Implementação com Python Dataclasses:

- Respeita naturalmente as restrições do problema.
- Reduz a complexidade computacional.

```
34 @dataclass  # Matheus Cascalho *
35 class solution:
36     ativo_equipe: list # h
37     equipe_base: list # y
38     fitness: float = 0
39     penalidade: float = 0
40
41     @property  # Matheus Cascalho
42     def fitness_penalizado(self):
43         return self.penalidade + self.fitness
44
45     @property  # Matheus Cascalho *
46     def ativo_base(self): # x
47         x = []
48         for equipe in self.ativo_equipe:
49             base = self.equipe_base[equipe]
50             x.append(base)
51         return x
52
```

Estruturas de vizinhança

- Refinamento local: vizinhanças 1 e 2;
- Grandes perturbações: vizinhanças 3 e 4.

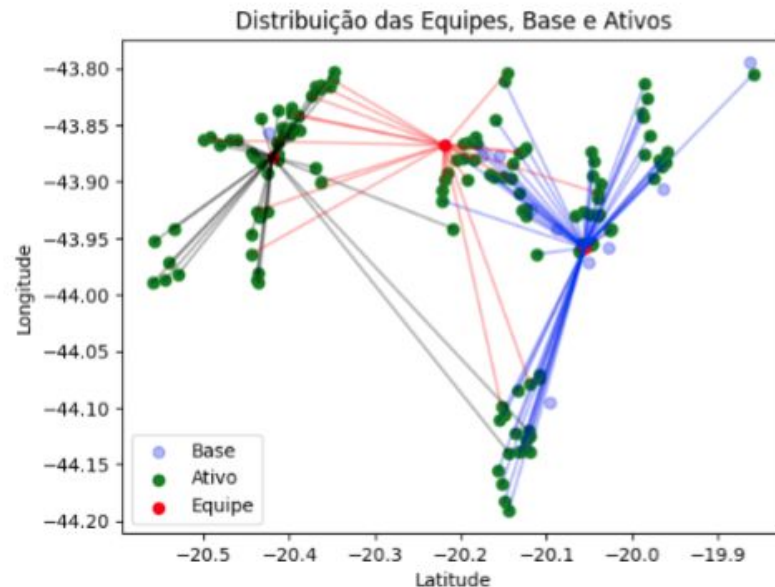
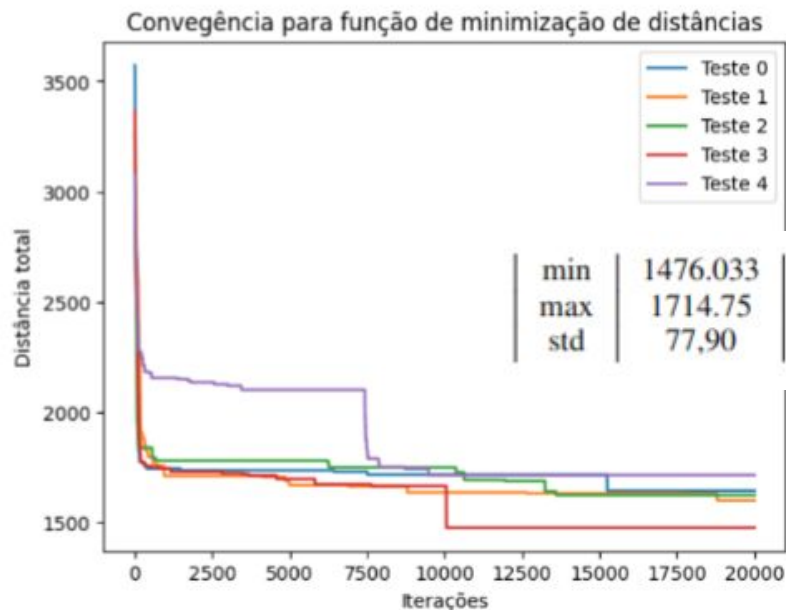
Algorithm 1: shake

Input: Solução Atual x , Índice de Vizinhança k ,
Problema $prob_def$

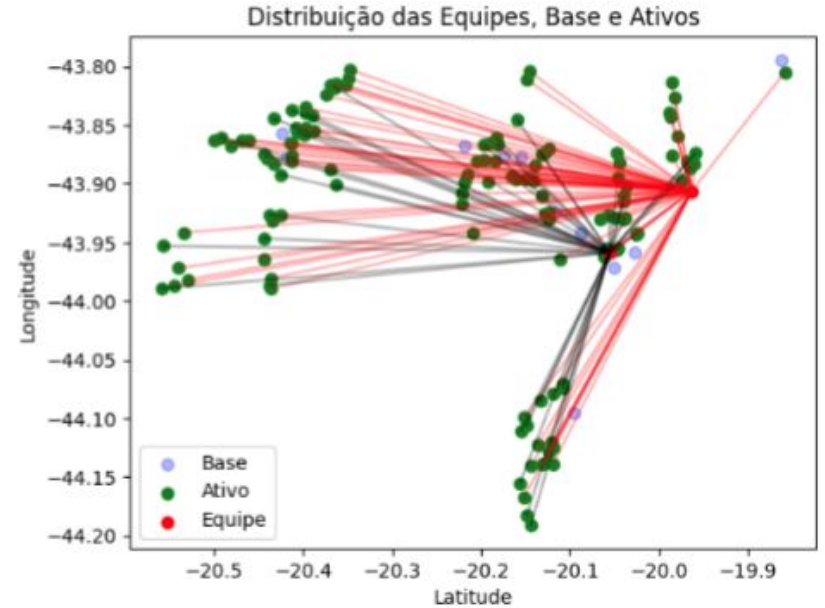
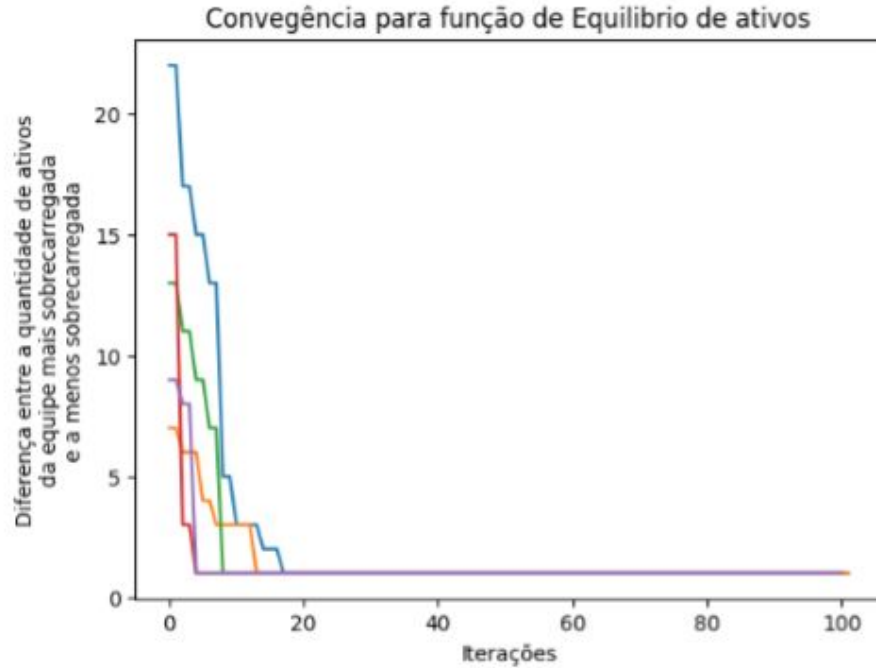
Output: Nova Solução y

```
1  $y \leftarrow$  Cópia de  $x$ ;  
2  $r\_equipe \leftarrow$  Índice Aleatório de Equipe;  
3  $r\_ativos \leftarrow$  Permutação Aleatória de Ativos;  
4 if  $k == 1$  then  
5   Alterar a base de  $r\_equipe$ ;  
6 else if  $k == 2$  then  
7   Alterar até 10 ativos aleatoriamente;  
8 else if  $k == 3$  then  
9   Alterar a base de  $r\_equipe$ ;  
10  Alterar até 30 ativos aleatoriamente;  
11 else if  $k == 4$  then  
12  Alterar todas as bases e até 30 ativos  
    aleatoriamente;  
13 return  $y$ ;
```

Convergência - Minimização de distância



Convergência - Equilíbrio de ativos



Conclusão

Metodologia eficiente para problemas reais

Limitações na robustez para escapar de ótimos locais

Melhorias futuras:

- Estratégias híbridas.
- Ajustes adaptativos nas vizinhanças.